# Advance Machine Learning

# Assignment: Text and Sequence

**Introduction**

This assignment takes a look at Recurrent Neural Networks (RNNs) as they see potential use in the processing and analysis of sequences for text data, focusing on sentiment analyses. This has been a focus of the project using the IMDB movie review dataset, a well-defined standard benchmark ever-used-for natural language processing tasks, to learn how these architectures do well under low data conditions, and how embedding methods potentially improve efficiency.

In this assignment, we will involve different configurations and analyze their performance, which includes reviewing cuts, limited training data, and even switching between learning and pre-trained embeddings. The hands-on experience supplements the course goal by bringing home the basic ingredients of deep learning, sequence modeling, and neural network architecture on how best to optimize performance when data availability is limited.

**Problem Statement**

Text data does not lend itself easily to machine learning's on-the-line-by-line treatment. It's sequential and high-dimensional from the start, making it almost moot for a lot of traditional models which fail too often in tasks like sentiment analysis because they're unable to hold context and temporal dependencies within the text sequences.

RNNs were thought up for this type of task so that it could encapsulate and use the sequential patterns. This project therefore exploits the abilities of RNNs to classify the sentiments of movie reviews from the IMDB dataset, under limited data conditions.

Some of the strategies include:

1. Truncating each review of a movie to maximum 150 words.

2. Searching the whole training dataset of only 100 samples.

3. Validation using 10,000 samples.

4. Limiting vocabulary to 10,000 of the most frequent words.

5. Compare performance of models with:a. standard learned embedding layer and b. pre-trained word embedding - both followed by a Bidirectional RNN.

**Primary Objectives**

This project is intended specifically to implement RNNs for sentiment analysis on the IMDB dataset, with a special emphasis on low-resource conditions. Specifically, the model developed should be geared toward effective learning despite very few training samples. Also evaluated is how different embeddings influence the overall accuracy and robustness of the model.

In this assignment, we hope to achieve the following:

- Find out how RNNs learn sequential dependencies in text.
- Evaluate the performance of pre-trained versus learned embeddings.
- Determine the minimum viable training data required for serious sentiment classification.

Ultimately, this project shall lead us in determining which preprocess methods and model settings are the most useful in improving performance on prediction, especially when limited by the amount of available data and silicon resources.

**Data Preparation**

The dataset that we use is the IMDB movie reviews dataset, which consists of 50,000 labeled reviews, either positive or negative. Several data pre-processing steps for a low-resource setting and robustness testing of models that characterize a little processing overhead have been instituted in this instance:

- Truncation: All reviews are truncated to a maximum of 150 words to ensure consistent input length and reduced processing overhead.
- Vocabulary Limitation: We limit our vocabulary to the 10,000 most frequently used words, helping reduce model complexity and avoid overfitting.
- Sample Constraints: The training set is capped at 100 samples, with 10,000 reviews set aside for validation purposes.
- Tokenization and Padding: Each review is tokenized into a sequence of integers, then padded to provide consistent input length for the model.

These preprocessing options are pivotal in saving computational efficiency and providing conditions to evaluate the performance of the RNN model in limited data conditions.

**Model Architecture**

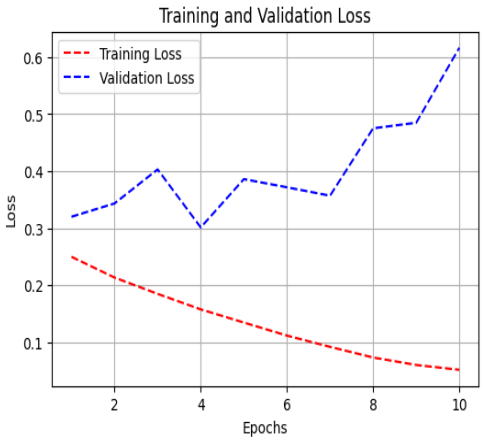
Fundamentally, the architecture of this project stands on a Bidirectional Recurrent Neural Network (BiRNN), which processes a text sequence in both forward and backward directions to provide better understanding of contexts. Bidirectional words are extremely helpful while performing sentiment analysis as the meaning of a word would often be different contextually with respect to the entire sentence.

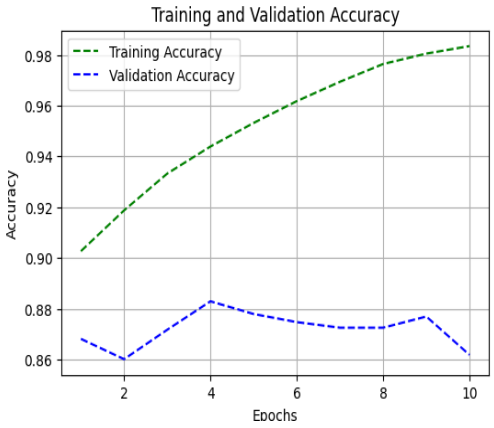
There were two independent implementations and evaluations of the model:

1. **Embedding Model:**

Embedding Layer: The model learns the word embeddings during the training process, which gives the model representation suited to the task.

Bidirectional LSTM: A layer reading the input sequence from both front and back in order to help the model capture context from both directions.



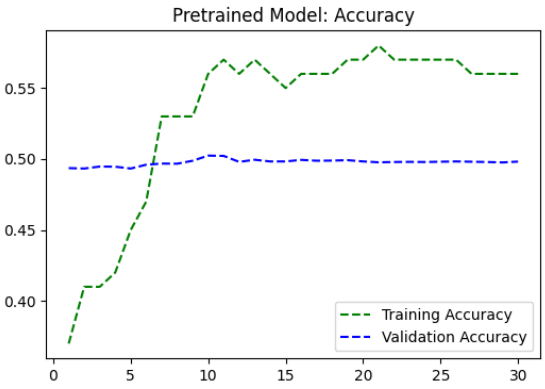



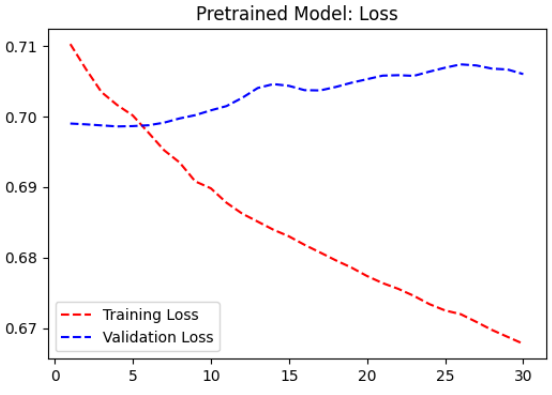

## 2. Pre-trained Embedding Model

Pre-trained Embedding Layer: This layer is initialized with word vectors such as GloVe that were trained on a large external corpus, endowing the model with a richer semantic understanding from the get-go.

Bidirectional LSTM: An identical architecture is adopted for the second model under which both models can be compared under the same testing conditions.

Dense Output Layer: A fully connected one that performs the final sentiment classification into positive or negative.



| Models Built | Training sample size | Validation size | Testing sample size | Model Type | Test accuracy (%) | Test loss |
|---|---|---|---|---|---|---|
| Model 1 | 100 | 10000 | 6000 | embedded layer | 51.2 | 0.693 |
| Model 2 | 10000 | 10000 | 6000 | embedded layer | 50.0 | 0.692 |
| Model 3 | 16000 | 10000 | 6000 | embedded layer | 89.4 | 0.251 |
| Model 4 | 10000 | 10000 | 6000 | one-hot | 86.1 | 0.360 |
| Model 5 | 16000 | 10000 | 6000 | LSTM using embedded layer | 51.3 | 0.693 |
| Model 6 | 32000 | 10000 | 6000 | LSTM using embedded layer | 93.8 | 0.174 |
| Model 7 | 10000 | 10000 | 6000 | masking enable | 48.8 | 0.693 |
| Model 8 | 100 | 10000 | 6000 | Pretrained using GloVe | 56.0 | 0.682 |
| Model 9 | 15000 | 10000 | 6000 | Pretrained with 4 LSTM hidden layers | 66.7 | 0.57 |
| Model 10 | 30000 | 10000 | 6000 | Pretrained with 2 LSTM hidden layers | 88.1 | 0.47 |

The purpose of this experiment was to compare models trained using two different types of word embeddings, specifically standard embedding layers, trained from scratch, and pretrained word embeddings, such as GloVe for the IMDB movie review dataset for the purpose of evaluating how these models perform under very different training sample sizes, especially in low-resource settings. The results indicated that with extremely little data-say, 100 samples-the model using pretrained embeddings performed better than the model using an embedding layer trained from scratch, with Model 8 obtaining an accuracy of 56.0% and Model 1 obtaining an accuracy of 51.2% using the GloVe embeddings and a simple embedding layer, respectively. This indicates the importance of pretrained embeddings when the model is barely able to learn enough word relationships due to scarcity of data.

However, as the training samples increased, those with learned embeddings turned out to be the better ones. In Model 3, for instance, the embedding layer with 16,000 training samples achieved an accuracy of 89.4%, while Model 6 with 32,000 returned an even larger increase in accuracy to 93.8%. These results mean that the moment a model has enough labelled data, it'll start nurturing task-specific word representations, which tend to overrule the general-purpose pretrained embeddings. Therefore, in low-data regimes, pretrained embeddings offer some help, but with sufficient data, embedding layers trained on the target task outperform them. This analysis thus gives us insight into the balancing act between the application of external semantic knowledge and the learning of task-specific embeddings depending on the availability of data.

**Conclusions**

The results of the study indicate that the use of pretrained or learned embedding layers depends on how much training data is available. By introducing semantic knowledge beforehand, pretrained embeddings such as GloVe can enhance performance when very few training samples are provided. While the amount of training data increases, models with embedding layers defined from scratch tend to outperform those that use pretrained vectors. More specifically, anything past about 16,000 training samples causes the learned embeddings to outperform significantly. Therefore, although pretrained embeddings provide a solid launchpad for low-resource environments, training specific embeddings is usually more useful when an adequate amount of data is available. These results suggest pertaining model design choices in relation to data availability for performance optimization in text-based deep learning tasks.